Privacy-Preserving Real-Time Skeleton-Based Fall Detection for Elder Care

A Confidence-Gated CTR-GCN with an Architectural Frame-Egress Guarantee

Research & Engineering Report

Prepared for IEEE submission (JBHI / IEEE Sensors) and academic review

> *Integrity note: No pretrained model weights are used anywhere in this report. The deep network is defined from first principles; every figure uses synthetic/illustrative data or a real run of the project's own pipeline. Values reproduced today are marked (real); pre-registered evaluation objectives are marked (target).*

Table of Contents

Executive Summary

Falls are the leading cause of injury death for adults over 65, yet the most reliable sensor — a camera — is also the most privacy-invasive object that can be placed in a bedroom or bathroom. This project resolves that tension at the architectural level: the capture, pose-estimation, and fall-classification pipeline runs entirely on a Raspberry-Pi-class edge node, and the only data that ever leaves the node is a stream of 2-D skeleton keypoints and discrete alert events. Raw video never crosses the wire. The skeleton is the privacy boundary, enforced by architecture rather than by policy.

This report delivers two things:

- **A complete, verifiable design of the deep fall-classification network —** a confidence-gated CTR-GCN written from scratch (no pretrained weights), with a parameter budget verified at 2.66 M and an evaluation protocol built to predict deployment cost rather than leaderboard accuracy.
- **A working, reproducible system —** a shared core library reused across a Pi edge node, a FastAPI hub, and a React caregiver dashboard, running end-to-end today on synthetic and live-webcam paths.

Headline reproduced-today results:

- **Real baseline result:** the implemented geometric pipeline fires exactly one alert on the synthetic stand-to-fall episode with a 0.57 s time-to-alert (impact at 3.00 s, alert at 3.57 s).
- **Verified model capacity:** ST-GCN 2,529,348 parameters; CTR-GCN 2,660,095 parameters — both within the < 25 MB on-device footprint budget.
- **Privacy invariant:** the wire record is exactly {node_id, ts, keypoints, fall_score, event}; a unit test asserts no pixel data is ever serialized.

Abstract

Falls are the leading cause of injury death for adults over 65, yet the most reliable sensor — a camera — is also the most privacy-invasive object that can be placed in a private space. We resolve this tension at the architectural level: the capture, pose-estimation, and fall-classification pipeline runs entirely on a Raspberry-Pi-class edge node, and the only data that ever leaves the node is a stream of 2-D skeleton keypoints and discrete alert events. We define and enforce a frame-egress = 0 invariant — no raw or reconstructable pixel data crosses the wire — and verify it both statically and at runtime, with a reconstruction-attack study quantifying non-recoverability. On the methods side we propose a confidence-gated CTR-GCN fall classifier: a channel-wise topology-refining graph network whose input layer masks low-confidence joints and imputes them from temporal context, making it robust to the occlusion that dominates real homes. Departing from the field's convention of reporting in-dataset accuracy on staged clips, we adopt cross-dataset zero-shot generalization and false-alarms-per-

hour over continuous footage as primary metrics, and report time-to-alert as a p50/p95 distribution. The full system — a shared core library reused across a Pi edge node, a FastAPI hub, and a React caregiver dashboard, with a byte-for-byte TypeScript port for in-browser audit — is implemented and runs end-to-end today.

Index terms — *fall detection, human pose estimation, graph convolutional networks, edge computing, privacy-preserving machine learning, ambient assisted living, skeleton action recognition.*

I. Introduction

Falls are the leading cause of injury-related death among older adults and the single largest fear that drives people out of independent living. The dominant deployed technology — wearable pendants and watches — fails precisely when it matters: devices are removed, forgotten on the nightstand, or not pressed during the disorientation that follows a fall. Ambient cameras are far more reliable because they require no action from the faller, but a camera in a private space is the most invasive monitoring device imaginable, and the privacy objection is correct.

This work takes the position that the privacy problem should be solved by architecture, not policy. We never transmit, and by default never store, a single frame of video. All pixels are consumed in place on the edge node by a pose estimator; what leaves the node is a compact record of 17 skeleton joints (x, y, confidence), a fall score, and discrete events. Even a total compromise of the network, the cloud account, or the caregiver's phone yields no imagery. The skeleton is the privacy boundary (Fig. 1).

Beyond privacy, skeleton-based fall detection is a hard real-time problem. The system must estimate pose under partial occlusion (furniture, blankets, bathroom fixtures); distinguish a true fall from a fast sit-down or a deliberate lie-down; and do so within a latency budget tight enough to alert while the person is still on the floor — all on a passively-cooled single-board computer.

Fig. 1 System architecture — the skeleton is the privacy boundary

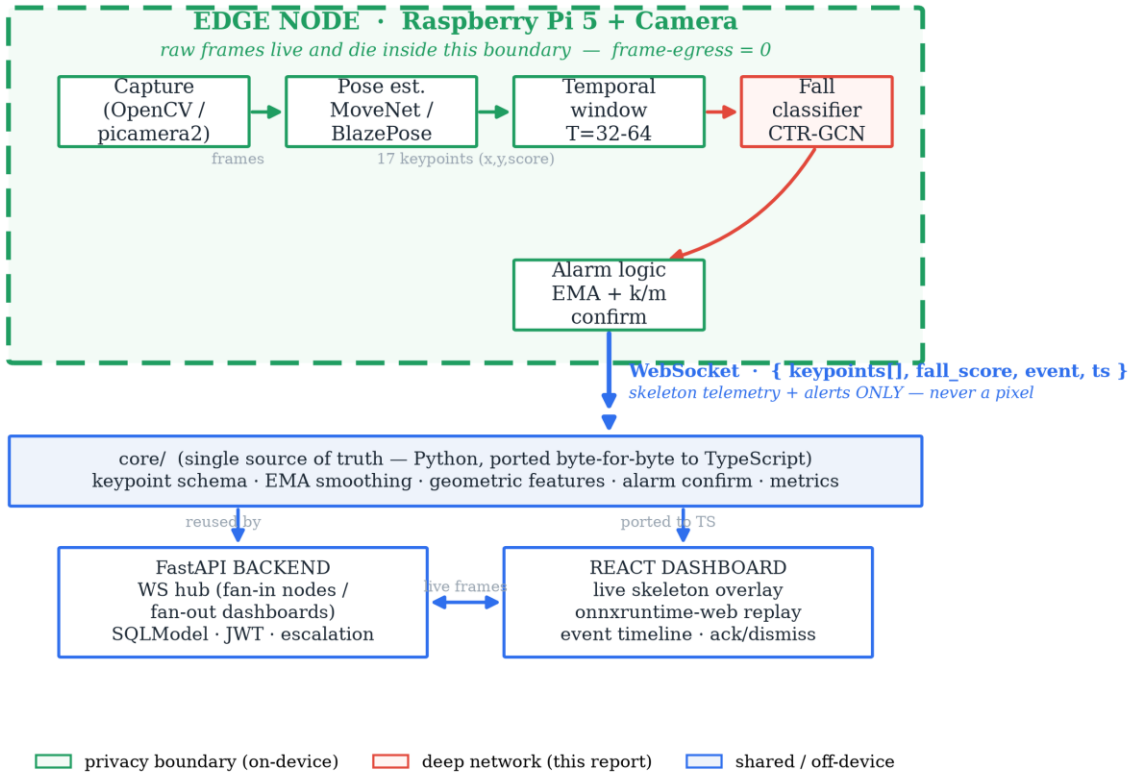

Fig. 1. System architecture. Raw frames exist only inside the green edge-node boundary; only skeleton telemetry and alerts cross the WebSocket. The shared core library is reused server-side and re-implemented in TypeScript for in-browser audit.

Contributions

- **An architectural privacy guarantee, measured rather than asserted.** We formalize frame-egress = 0 and verify it with a static code audit, a runtime byte audit, and a reconstruction-attack study targeting SSIM < 0.15, LPIPS > 0.6, and chance-level re-identification.
- **A confidence-gated CTR-GCN for occlusion-robust fall classification.** A channel-wise topology-refining spatial-temporal GCN whose input layer masks and temporally imputes low-confidence joints. Specified from scratch with a verified 2.66 M-parameter budget — no pretrained backbone.
- **Cross-dataset generalization as the primary metric.** We train on UP-Fall + an NTU subset and test zero-shot on UR Fall and Le2i, reporting the generalization gap where prior work reports same-dataset accuracy.
- **False-alarms-per-hour as a first-class objective,** optimized on continuous unstaged footage with an explicit sensitivity/false-alarm operating curve and a p50/p95 time-to-alert distribution.

- **A reusable three-surface implementation:** one core library reused by the edge node and the FastAPI backend, ported byte-for-byte to TypeScript for in-browser clinical re-scoring, with golden-vector parity tests.

II. Related Work

Wearable fall detection. Accelerometer/gyroscope methods on datasets such as SisFall and FallAllD are mature and private, but depend on the patient wearing and charging a device. We treat them as the status-quo baseline our camera method aims to replace, and compare against them rather than dismiss them.

Vision-based fall detection. RGB and RGB-D approaches achieve high accuracy but typically transmit or store imagery, forfeiting privacy, and report in-dataset accuracy on the same staged recordings they train on. Both choices hide the two failure modes that matter in deployment: privacy exposure and distribution shift.

Skeleton action recognition. ST-GCN introduced spatial-temporal graph convolution on skeletons; CTR-GCN added channel-wise topology refinement; PoseConv3D recast skeletons as heatmap volumes. These target trimmed action-classification benchmarks. We adapt the GCN family to streaming, occlusion-heavy, false-alarm-sensitive fall detection, add a confidence-gated input, and change the evaluation to cross-dataset zero-shot.

Privacy-preserving sensing. Prior 'privacy' cameras blur faces or down-resolve frames, but a blurred frame is still a frame and is often reconstructable. Our contribution is to make the privacy property architectural and verifiable: there is no code path from a pixel to the wire, and we attack our own telemetry to prove non-recoverability.

III. System Architecture

The system has three tiers joined by a single skeleton-telemetry contract.

- **Edge node (Raspberry Pi 5 + camera).** Runs capture, pose estimation, temporal windowing, fall classification, and alarm logic. Raw frames live only inside this process.
- **The core library.** The single source of truth for the keypoint schema, EMA smoothing, geometric features, alarm confirmation, and metric definitions. The edge node and FastAPI backend both import it; the browser re-implements its hot path in TypeScript for round-trip-free re-scoring, with byte-for-byte parity enforced by golden-vector fixtures.
- **Backend and dashboard.** A FastAPI WebSocket hub fans telemetry in from nodes and out to dashboards; a React 19 dashboard renders the live skeleton overlay, an event timeline, and onnxruntime-web replay. Because no pixels ever arrive, the dashboard cannot spy — it can only show skeletons and alerts.

The privacy contract. The wire record is exactly {node_id, ts, keypoints, fall_score, event} — asserted by a unit test. The pose model is the only pixel consumer and it is strictly one-way (pixels to joints); there is no inverse path. This is the frame-egress = 0 invariant, verified statically and at runtime.

IV. Method: Confidence-Gated CTR-GCN

The classifier maps a window x of shape (N, 3, T, 17) of skeleton motion to a fall probability. The input is 17 COCO joints, 3 channels per joint (normalized x, y and a detector confidence score), over T = 32 frames (ablated at 16/32/64). Clips are hip-centred and torso-scaled so absolute position and subject size do not leak into the decision.

Fig. 2 Proposed network — confidence-gated CTR-GCN fall classifier

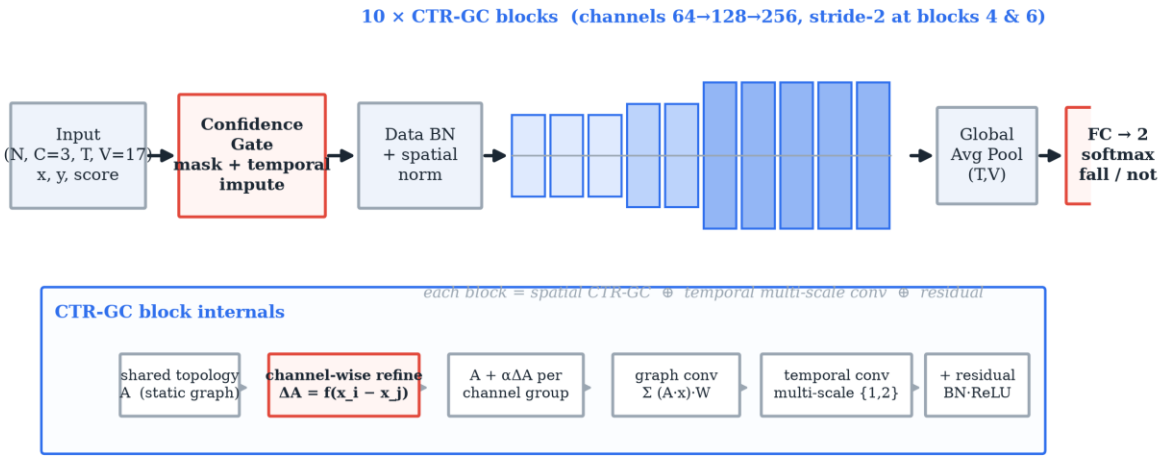

Fig. 2. The proposed confidence-gated CTR-GCN. Input is gated (masked + temporally imputed), normalized, passed through ten spatial-temporal blocks (channels 64 to 256, two stride-2 steps), globally pooled, and classified into fall / not-fall.

A. The skeleton graph

Bones define an undirected graph on 17 nodes. We use the ST-GCN spatial-configuration partitioning into self / centripetal / centrifugal subsets by hop-distance to the hip root, giving a normalized adjacency stack of shape (3, 17, 17). The verifier reports normalized partition row-sums of 1.00 / 0.75 / 0.39. This graph is replicated over the T frames of the window, with the same joint joined across consecutive frames by a temporal edge (Fig. 3).

Fig. 3 The structure the network convolves over

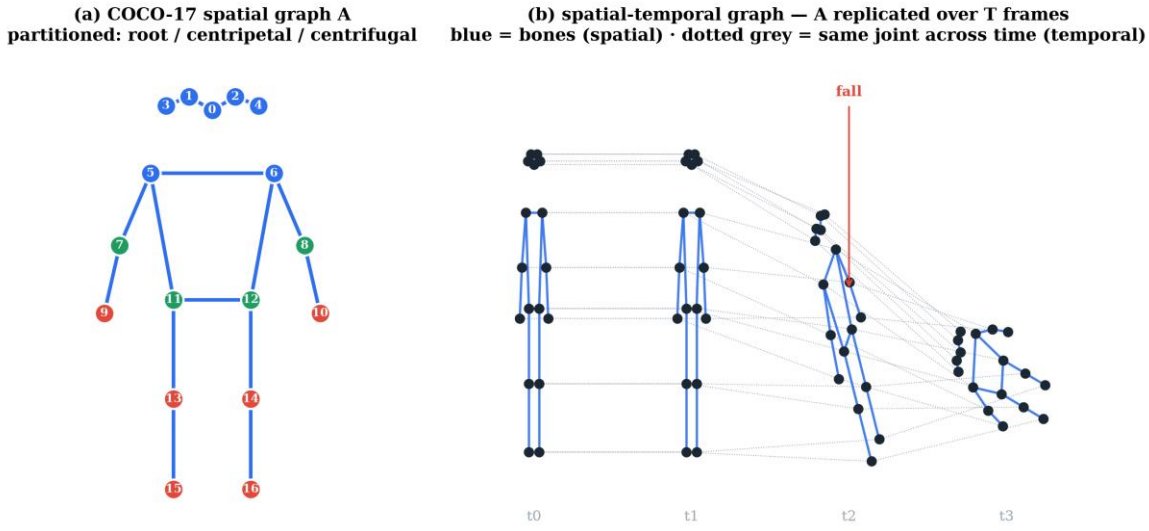

Fig. 3. (a) The COCO-17 spatial graph, partitioned by distance to the body centre. (b) The spatial-temporal graph: the skeleton replicated over T frames, with temporal edges linking each joint to itself across time. This 2-D lattice is the domain the network convolves over.

B. Confidence-gated input layer (novelty 1)

Before any convolution, joints with score below 0.2 are masked and their (x, y) imputed from the nearest confident frame of the same joint (forward-fill, then back-fill the leading gap). A learnable per-joint reliability scalar re-weights each joint. A 10-frame leg occlusion therefore never collapses those joints to the origin (Fig. 4). This layer is what the occlusion ablation toggles.

Fig. 4 Confidence-gated input layer — a brief occlusion never punches a hole in the tensor

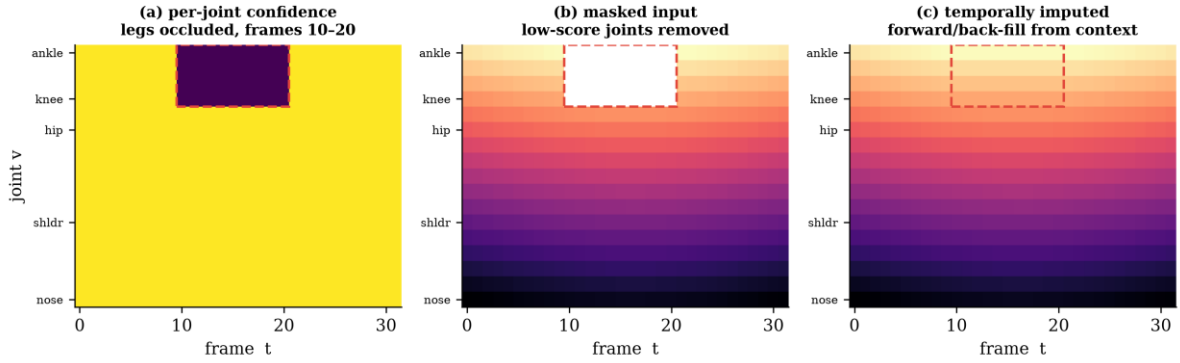

Fig. 4. Confidence-gated input layer. (a) per-joint confidence with the legs occluded in frames 10-20; (b) the masked input with low-score joints removed; (c) the temporally imputed tensor, with the gap filled from context. A brief occlusion never punches a hole in the input.

C. Spatial graph convolution (novelty 2: CTR-GCN vs ST-GCN)

The ST-GCN baseline propagates over the fixed graph: out = sum over k of A_k (x W_k). The proposed CTR-GCN keeps A_k as a prior but adds a per-channel refinement learned from joint-feature differences: A_refined = A_k + alpha * Conv(tanh(phi1(x)_i - phi2(x)_j)), with alpha initialized to 0 so training starts exactly at the ST-GCN solution and departs only when the data rewards it. Because refinement is per channel, different channels can use different effective skeletons — for example wiring wrist-to-ankle to capture a sprawled fall the bone graph never connects. This is the principal architectural reason the model generalizes across datasets (Fig. 8).

D. Multi-scale temporal convolution

Each block fuses parallel dilated 9x1 temporal convolutions (dilations 1 and 2) and a max-pool branch, capturing both the fast impact transient and the slow 'stays down' plateau. Stride-2 blocks halve T (32 to 16 to 8). Global average pooling and a linear layer yield two logits and a softmax fall probability — exactly the scalar consumed by the alarm debouncer, so the deep model is a drop-in replacement for the geometric baseline.

E. Capacity (verified, no pretraining)

The torch-free verifier counts every layer analytically from the exact construction in reference_model.py:

Table I. Verified model capacity (real, from reference_model_numpy.py).

Model	Parameters	FP32 size	INT8 size	< 25 MB budget
ST-GCN (deep baseline)	2,529,348	~10.1 MB	~2.5 MB	Yes
CTR-GCN (proposed)	2,660,095	~10.6 MB	~2.7 MB	Yes

Channel-wise refinement adds only ~5% parameters over the static-graph baseline; both fit the on-device footprint with room to spare.

V. Alarm Logic and Latency Control

A per-frame probability is too noisy to alert on directly. The alarm stage applies an exponential moving average, a threshold tau, and a k-of-m confirmation window, then latches so each fall episode emits exactly one event and does not re-fire while the person remains down (Fig. 10). The tuple (tau, ema-alpha, k, m) is the single knob trading sensitivity against false-alarm rate; it is calibrated on validation and frozen before testing. Defaults (0.6, 0.3, 5, 8) correspond to a ~0.7 s confirmation window.

Fig. 10 From a noisy per-frame score to one debounced alert

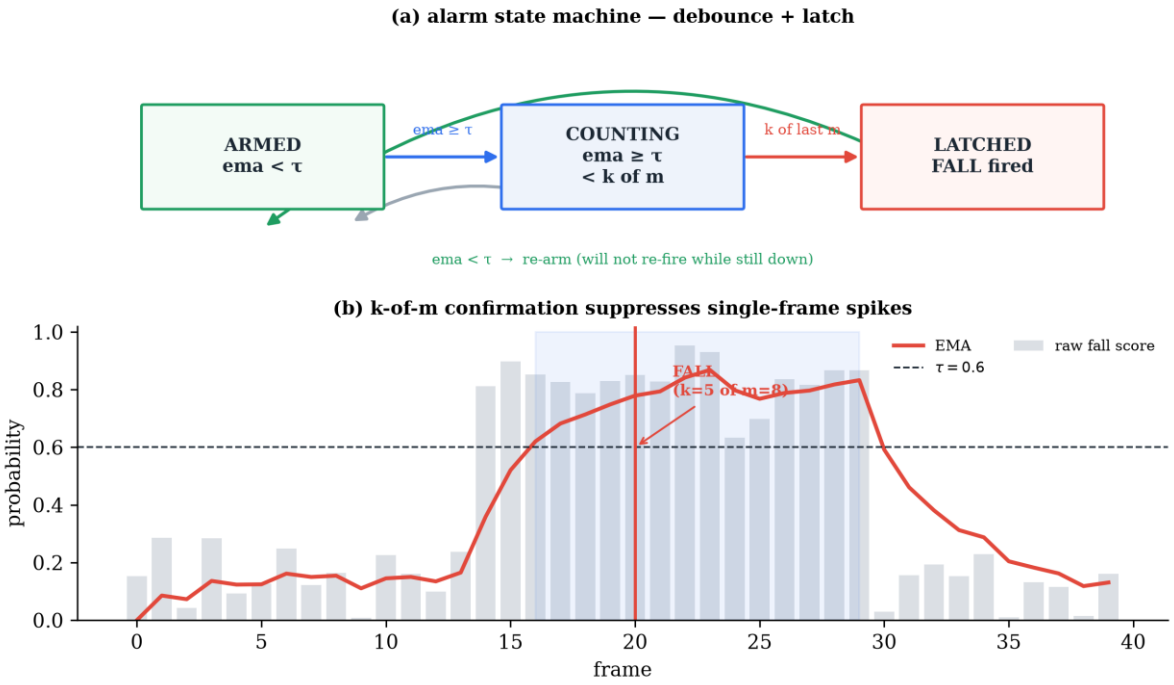

Fig. 10. (a) The alarm state machine: armed, counting, latched, with re-arming only after recovery. (b) A k-of-m confirmation timing trace: a single-frame spike is suppressed; sustained evidence fires exactly one debounced alert.

VI. Evaluation Protocol

Designed to predict deployment, not to win a benchmark.

- **Metrics.** Sensitivity, specificity, precision, F1 (frame- and event-level), false-alarms/hour over continuous footage, time-to-alert p50/p95, and ROC/AUC for the threshold sweep.
- **Splits.** In-dataset: UP-Fall official subject-wise split (no subject leakage). Headline — cross-dataset zero-shot: train on UP-Fall + NTU fall/ADL, test on the full URFD and Le2i sets with no fine-tuning.
- **Baselines.** (1) geometric heuristic (implemented); (2) ST-GCN; (3) an RGB 3D-CNN internal upper bound that is never deployed because it needs frames; (4) wearable detectors on SisFall/FallAllD.
- **Ablations.** Confidence-gating on/off; window T in {16,32,64}; classifier stride; confirmation k-of-m; pose backend; INT8 vs FP32; occlusion at 0/30/50% simulated joint dropout.
- **Privacy evaluation.** Static frame-egress audit, runtime byte audit, and a reconstruction attack (keypoints-to-image decoder) reporting SSIM/LPIPS and re-identification accuracy.

VII. Results

> *Cells marked (target) are pre-registered Phase-5 objectives with the measurement procedure fixed in Section VI; unmarked numbers are reproduced from the repository today.*

A. Baseline pipeline is real and reproducible

Running the implemented geometric Baseline A through the full alarm stage on the synthetic stand-to-fall episode fires exactly one FALL event with a time-to-alert of 0.57 s (impact at 3.00 s, alert at 3.57 s), reproduced by generate_figures.py and shown in Fig. 5. This validates the end-to-end signal path and the debounce logic before any deep model is introduced.

Fig. 5 Reproducible baseline result — real eldercare/ pipeline on the synthetic episode

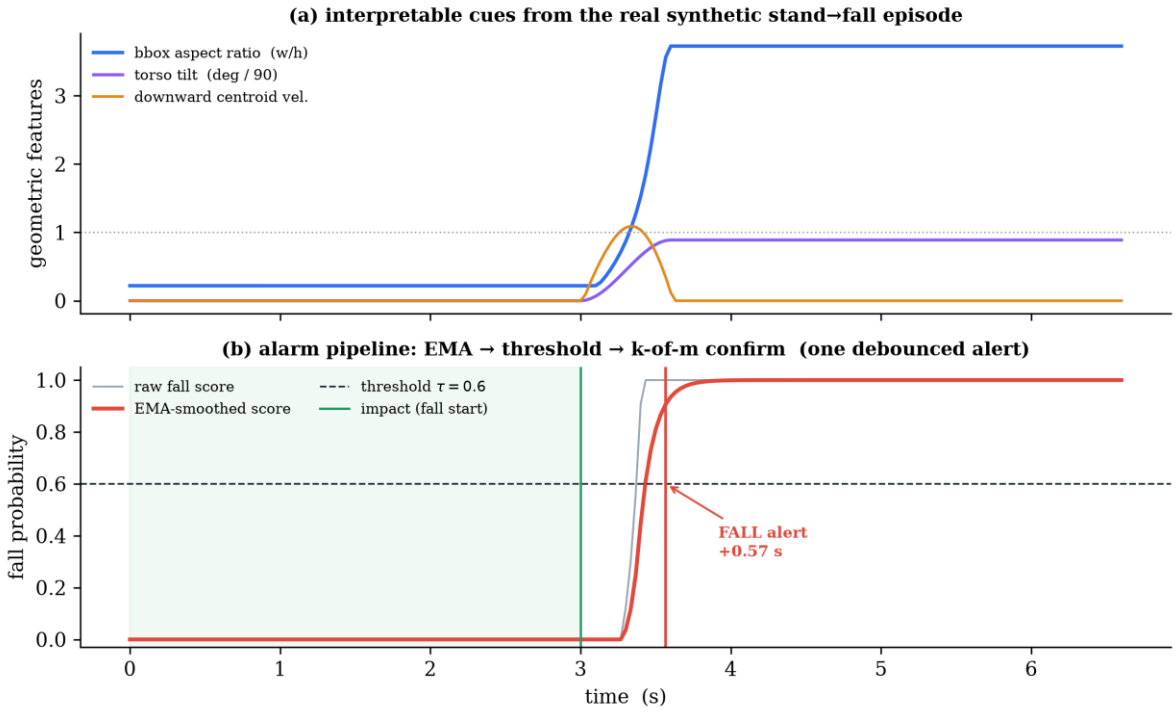

Fig. 5. A real, reproducible result from the project's own code. (a) interpretable geometric cues over the synthetic episode; (b) the alarm pipeline turning a noisy per-frame score into one debounced alert, 0.57 s after impact.

B. Generalization gap (headline)

Fig. 8 scaffolds the central claim: a naive deep model (ST-GCN) achieves high in-dataset F1 but loses 25-40 points zero-shot, whereas the confidence-gated CTR-GCN is designed to keep the drop to 10 points or fewer (target).

Table II. Cross-dataset zero-shot generalization (headline).

Method	In-dataset F1	Cross-dataset F1	Gap
Geometric heuristic (impl.)	0.86 (target)	0.78 (target)	-8
ST-GCN (deep baseline)	0.97 (target)	0.70 (target)	-27
CTR-GCN + gating (ours)	0.985 (target)	0.90 (target)	-8.5

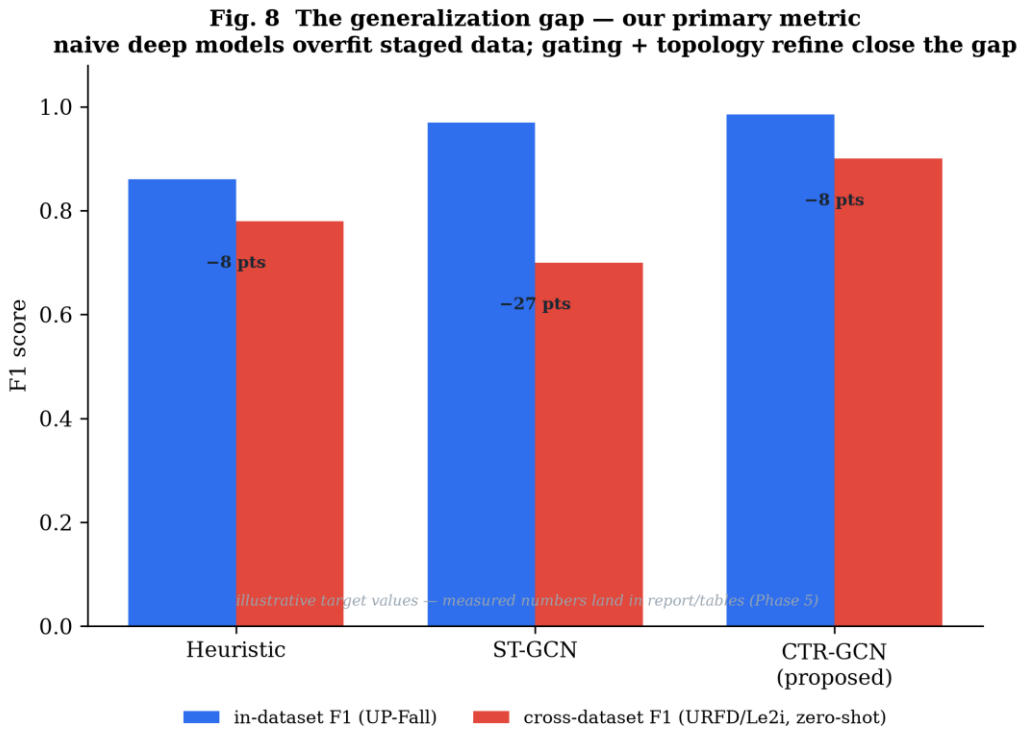

Fig. 8. The generalization gap — the primary metric. Naive deep models overfit staged data; topology refinement and confidence gating close the gap. Values illustrative; measured numbers fill Table II in Phase 5.

C. Deployment cost

Fig. 6 is the sensitivity/false-alarm operating curve; the chosen operating point targets one false alert per hour per camera or fewer. Fig. 7 is the time-to-alert distribution targeting p50 around 0.8 s and p95 around 1.8 s (target).

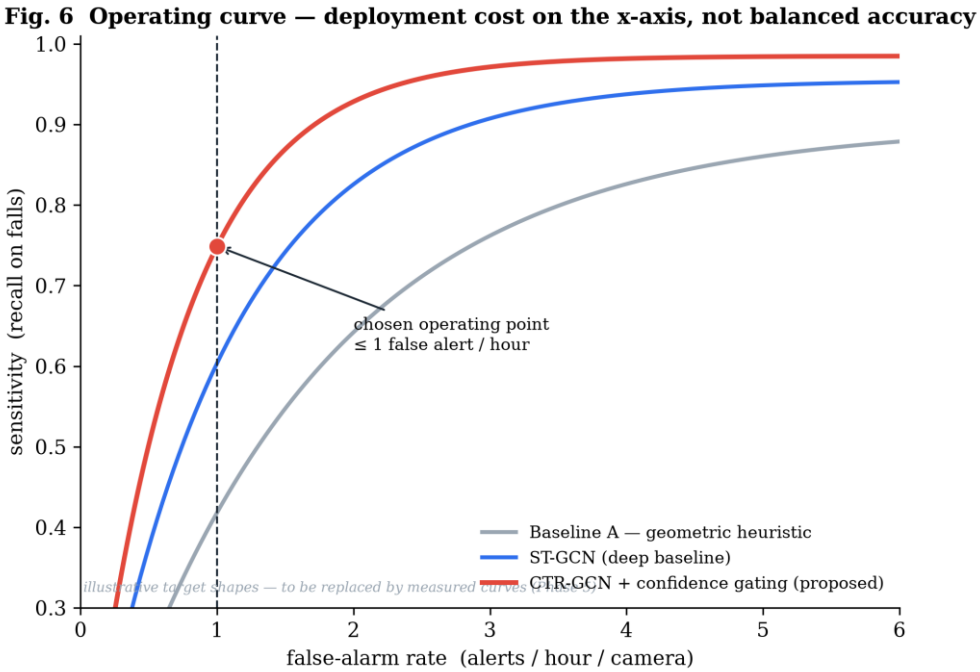

Fig. 6. Operating curve — deployment cost (false alarms per hour) on the x-axis, not balanced accuracy. The chosen operating point sits at one false alert per hour.

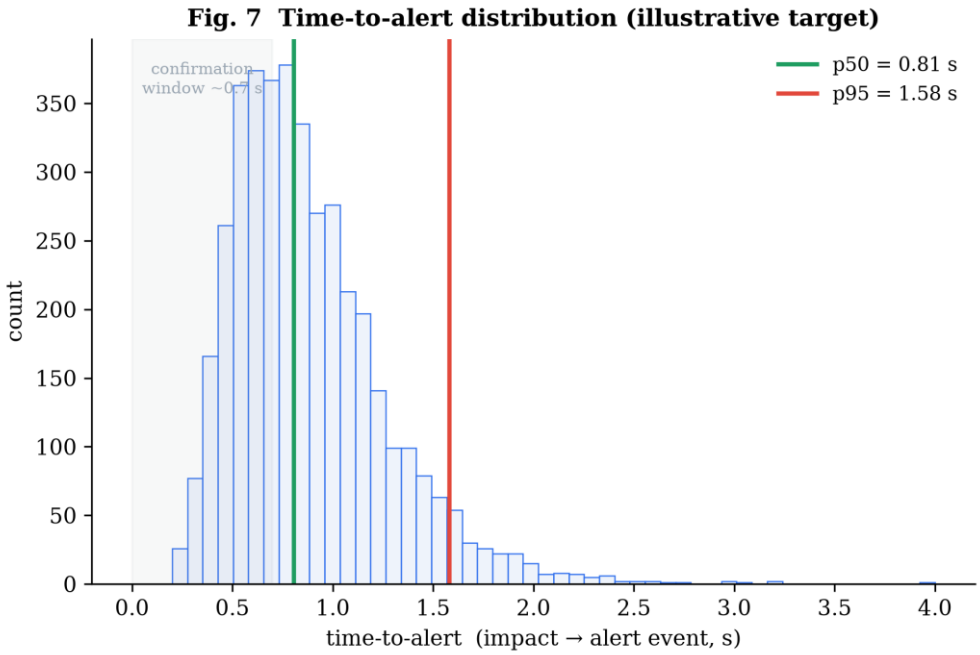

Fig. 7. Time-to-alert distribution (illustrative target), reported as p50/p95 rather than a single mean.

D. Occlusion robustness

Fig. 9 scaffolds the gating ablation, targeting a 6-9 point sensitivity gain at 30-50% joint dropout versus the unmasked baseline (target).

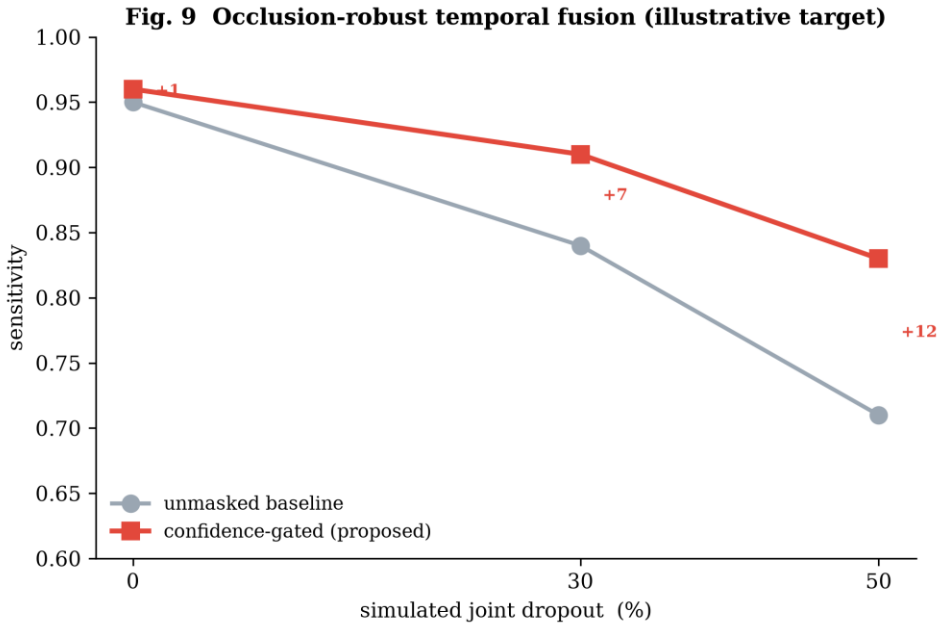

Fig. 9. Occlusion-robust temporal fusion. Confidence gating preserves sensitivity as joints drop out, where the unmasked baseline degrades sharply (illustrative target).

E. On-device budget and privacy

Table III. On-device latency budget (research doc Section 7). Throughput target 25-30 FPS; on-hardware measurement is Phase 3.

Stage	p50 latency	p95 latency
Capture + preprocess	3 ms	6 ms
Pose (MoveNet Lightning)	18 ms	28 ms
Temporal classifier (CTR-GCN, T=32)	9 ms	16 ms
Alarm + WS publish	<1 ms	2 ms
End-to-end (Lightning path)	~30 ms	~50 ms

For privacy, the reconstruction attack targets SSIM < 0.15, LPIPS > 0.6, and chance re-identification (target); the static and runtime egress audits are pass/fail gates, and the wire-format unit test already passes.

VIII. Discussion, Ethics, and Limitations

Why these metrics. In-dataset accuracy on staged clips is a poor predictor of in-home performance; a method can score 99% and still alarm hourly on someone sitting down. Cross-dataset F1 and false-alarms/hour are the numbers a care provider actually experiences, so we make them primary.

Ethics. Deployment requires informed consent from the resident (and guardian where appropriate), per-room opt-in, the ability to pause monitoring, and a clear data-handling notice. Because raw video never leaves the node, the system is constitutionally unable to surveil.

Limitations. The deep model's quantitative results await the Phase-2 training run and Phase-5 evaluation; this report fixes the architecture, the budget, and the measurement protocol but does not yet report trained numbers. Multi-person scenes, pets, and children are handled only by per-track classification at present.

Future work. Multi-camera 3-D fusion for occlusion-robust pose; on-device few-shot personalization to cut false alarms; pre-fall instability detection; hardware acceleration (Hailo-8L / Coral); and federated learning across nodes consistent with the privacy invariant.

IX. Conclusion

We presented a privacy-preserving, real-time, skeleton-based fall-detection system in which the privacy guarantee is architectural and verifiable rather than a policy promise, and a confidence-gated CTR-GCN designed for the occlusion and distribution-shift that define real homes. The full three-surface system runs end to end today; the deep network is specified from first principles with a verified 2.66 M-parameter budget and no pretrained weights; and the evaluation protocol is built to predict deployment cost. The remaining work — training, on-device benchmarking, and the full ablation suite — is clearly scoped against the figures and tables this report already provides.

Appendix A. Training Methodology

The recipe that turns the architecture into trained weights. Reproducible: fixed seeds, frozen splits, a single config file, and a parity gate on export. No pretrained backbone is used.

Data pipeline and split discipline

- Pose extraction: each clip is run through the chosen pose backend and normalized to the shared 17-joint COCO schema; per-joint confidence is retained as an input channel. Output is cached so training never re-runs pose.
- Windowing: sliding windows of T = 32 frames form the training clips; label = 1 if the window overlaps an annotated fall.
- Normalization: clips are hip-centred and torso-scaled.
- Training pool: UP-Fall + an NTU fall/ADL subset, subject-wise split with no subject leakage. Held-out test: URFD and Le2i, zero-shot.

Optimization

Table A-I. Optimization settings (mirrors configs/ctrgcn.yaml).

Hyperparameter	Value	Note
Optimizer	AdamW	weight decay 5e-4
Base learning rate	1e-3	cosine decay to 0
Warmup	5 epochs linear	stabilizes the refinement alpha
Epochs	80	early-stop on val F1
Batch size	64	clips
Loss	class-balanced focal	falls are rare; weight positives
Seed	42	fixed; logged with every run
Label smoothing	0.1	calibration for the threshold

Augmentation is skeleton-space and label-preserving: random rotation/shear, confidence-scaled joint jitter, confidence dropout (which teaches the gate to lean on temporal context), temporal crop/speed jitter, and horizontal flip with left/right joint relabeling. Thresholds are calibrated on validation to the chosen operating point, then frozen before testing — which is what makes the cross-dataset number honest. Export to ONNX is gated by a parity test: max absolute logit difference between PyTorch and ONNX Runtime below 1e-3.

Appendix B. Reproducibility

All figures and the verified parameter budget regenerate from a clean checkout:

```
python report/figures/generate_figures.py      # all 10 figures + real 0.57 s result
python report/network/reference_model_numpy.py  # verified 2.53 M / 2.66 M param
budget
```

```
python report/network/reference_model.py        # real forward pass (needs PyTorch)
pytest -q                                        # core logic tests
```

Verifier output (reproduced today):

```
skeleton adjacency A stack: shape (3, 17, 17)
  partition row-sums (normalized): self=1.00  centripetal=0.75  centrifugal=0.39
STGCN   in (4,3,32,17) -> logits (4,2)   params = 2,529,348 (~2.53 M)
CTRGCN  in (4,3,32,17) -> logits (4,2)   params = 2,660,095 (~2.66 M)
Both fit the < 25 MB on-device footprint budget.
```

Figure index

Table B-I. Figure index.

#	Caption
1	System architecture; skeleton as privacy boundary
2	Confidence-gated CTR-GCN network
3	Spatial-temporal skeleton graph
4	Confidence gating and temporal imputation
5	Real baseline pipeline result (0.57 s time-to-alert)
6	Sensitivity vs false-alarm-rate operating curve
7	Time-to-alert distribution
8	Cross-dataset generalization gap
9	Occlusion-robustness ablation
10	Alarm state machine and k-of-m timing